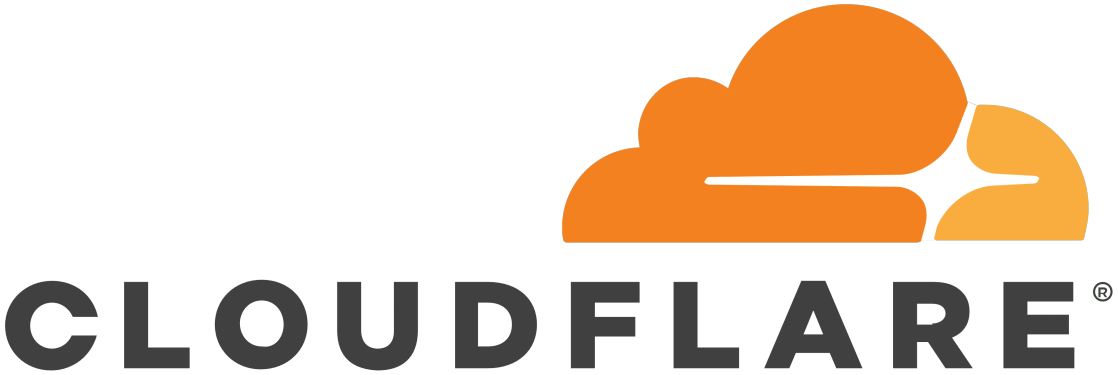


Image Gallery Platform – Implementation Report

March 17, 2026

1 Summary

This project implements a globally distributed image gallery API using Cloudflare's developer platform services. It securely stores images, delivers them through the Cloudflare edge network, and automatically enriches them with descriptive **Alt-Text** generated by AI vision models, in case it was not initially provided.

The solution is currently hosted in image-gallery.goncalo2k.pt and its Github repo is under goncalo2k/image-gallery.

The solution is designed to improve **performance, accessibility, and operational efficiency** while minimizing infrastructure costs and complexity. Images are stored in **Cloudflare R2**, delivered through **Cloudflare Workers**, and enriched using **Workers AI Image-to-Text models**. Generated descriptions and metadata are persisted in a **D1 database**, ensuring that each image is processed only once and avoiding unnecessary costs when processing images. The requests for specific images are cached and invalidated uppon image deletion using **Cloudflare's Cache API**, which allows frequently requested images to be served directly from edge locations around the world, therefore optimizing performance and reducing compute usage.

Additionally, the system includes administrative and operational capabilities such as:

- An **audit endpoint** for inspecting processed images and their metadata
- **Dynamic image ingestion** from external URLs
- **Image ingestion** from local files
- Delete images previously uploaded
- List all images currenty available
- Secure service-to-service communication between storage, compute, and database layers thanks to Worker Bindings

2 Solution Requirements

The initial requirements presented for this product were the following:

2.1 Business Requirements

- The platform must provide a scalable and globally accessible image delivery service capable of serving images with descriptive metadata to improve accessibility.
- The system must automatically generate and manage descriptive alt-text for images to enhance accessibility compliance and improve user experience.
- The platform must minimize operational costs by leveraging edge caching and efficient AI inference usage.
- The system must maintain a persistent and auditable record of processed images and their associated metadata.
- The platform must support automated ingestion of new image assets to streamline content management workflows.
- The system must enforce secure storage and controlled access to image assets and administrative interfaces.

2.2 Functional Requirements

- Image Storage: The system must initialize a private Cloudflare R2 bucket and it must be populated with a small set of sample images for testing.
- Edge Image Delivery: A Cloudflare Worker must serve images directly to web browsers. Each image response must include a descriptive alt-text string in a custom HTTP header.
- Alt-Text Generation: The system must generate image descriptions using Workers AI vision models when alt-text is not available. This should be done while respecting Workers AI daily neuron limits. The system must avoid re-running inference for images that have already been processed.
- Persistent Metadata Storage: A Cloudflare D1 database must store generated alt-text descriptions and associated image metadata. The system must query the database to determine whether an image has already been processed.
- Caching and Performance: The system must leverage Cloudflare's Cache API for performance and compute optimization. The architecture must also leverage non-blocking background operations so image delivery is not delayed by database or AI processing.
- Administrative Endpoint: A secure `/audit` endpoint must return a JSON list of processed images and their stored metadata from D1.

- Dynamic Image Ingestion: The system must allow new images to be ingested from an external URL, storing them in the R2 bucket and automatically triggering description generation and metadata storage.

2.3 Security Requirements

- Secure Storage: The R2 bucket must remain private and inaccessible directly from the public internet.
- Endpoint Protection: Administrative endpoints (e.g., `/audit`) must require secure access controls.
- Environment Security: Secrets (API keys, tokens, database credentials) must be stored using Cloudflare environment variables or secret management.

3 Real Use Cases

This API was designed to be used by other services in a variety of ways. Possible use cases are:

1. **Editorial portfolio backend** – The Worker powers a personal portfolio that cycles through stored images. A back-office or CMS jobs call `/images/external` with curated URLs or actual files, the Worker stores binaries in R2, generates Workers AI alt text, and exposes metadata via `/images` and `/images/:id`. The BFF API that powers the Frontend consumes the `/images` and `/images/:id` endpoints to get the images and serves them to the Frontend, for them to be dynamically displayed in a caroussel, while leveraging the caching at the Edge level.
2. **Internal design library** – A design team mirrors approved brand assets by uploading local files through `/images`. D1 metadata acts as the source of truth for descriptions, while `GET /images/audit` feeds a Notion dashboard that tracks total assets and most recent updates, from which the users can select images and get redirected to the `/images/:id` endpoint, that serves the selected image alongisde with its alt-text.

4 Customer Experience

The focus of this API is to fulfill the systems' requirements without compromising the developer experience. While developing this product, there was a big emphasis on producing a smooth experience for systems and developers utilizing this API.

- **Fast delivery** – Images stream from Cloudflare's edge with cache hits served immediately and cache misses still benefitting from R2's proximity to Workers. Predictable latency keeps portfolio visitors and internal stakeholders engaged.

- **Accessible by default** – Every response exposes descriptive text through `ALT_HEADER_NAME`, so consuming apps can provide meaningful alt text without a separate metadata fetch.
- **Self-service operations** – The REST surface is minimal (`/images`, `/images/:id`, `/images/audit`), and the OpenAPI schema + Swagger UI give non-engineers an explorable contract. Scripts handle common chores (bulk upload/clear), reducing toil for content teams.
- **Secure touchpoints** – Cloudflare Access service tokens, CORS allow-lists, and timing-safe credential checks ensure that only trusted systems can mutate the gallery, while public endpoints (`/health`, `/docs`, `/openapi.json`) remain open for observability.

Setup Instructions

4.1 Local development:

4.1.1 Prerequisites:

1. Install PNPM;
2. All the intended Cloudflare Products created. If there is the need to understand how to do this, jumping to section 7.1 is recommend;

4.2 Local Setup

1. Clone the github repo
2. Install dependencies using `pnpm i`
3. Update `wrangler.jsonc` to contain the desired `routes` and `custom_domains`, the bindings for `d1_databases` and `r2_buckets` as well as the Worker's `name`. Note that you can use the database and bucket bindings with local or remote versions.
4. Copy `.dev.vars.example` and rename it to `.dev.vars`, updating the values inside.
5. For local D1 setup, run `pnpm run setup-local-db`.
6. (Optional) To populate the database and the blob storage, use `pnpm run batch-upload`.
7. (Optional) To purge both D1 and R2, run `pnpm run clear-images`.
8. Run `pnpm run cf-typegen` to generate types.
9. Then, just run the Worker with `pnpm dev`

Note that, on every commit, both `pnpm lint` && `pnpm test` are run to ensure nothing is broken.

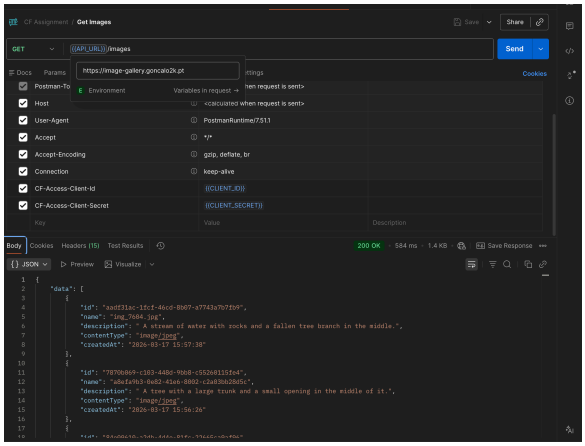
5 Instructions to Use the Platform & Testing Evidence

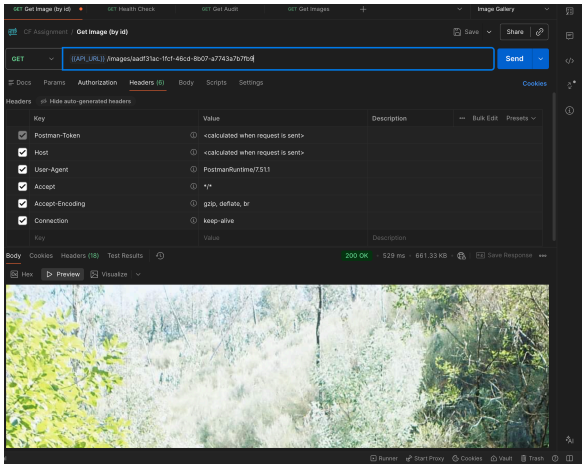
Note that all endpoints work exactly the same in the deployed version and in the local version and to test locally, running the worker on your local machine and changing from `https://image-gallery.goncalo2k.pt` to `http://localhost:8787`. However, the deployed `/audit` has an extra layer of security due to the **Access Policy** previously defined and the **401 Response for Service Auth** being enabled. Also note that all the endpoints are present in the postman collection under the folder `postman` of the repo.

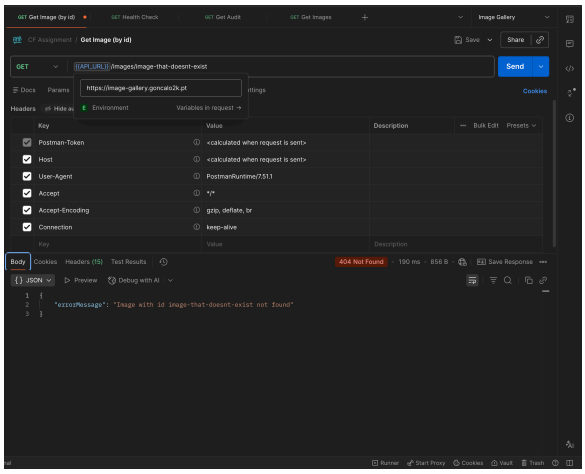
5.1 Viewing Images

Hit `GET https://image-gallery.goncalo2k.pt/images` to retrieve paginated metadata (`offset`, `limit` query params). Fetch an individual asset with `GET https://image-gallery.goncalo2k.pt/images/id`; successful responses include `Content-Type`, cache headers, and `ALT_HEADER_NAME` for instant consumption in web browsers.

5.1.1 Testing Evidence



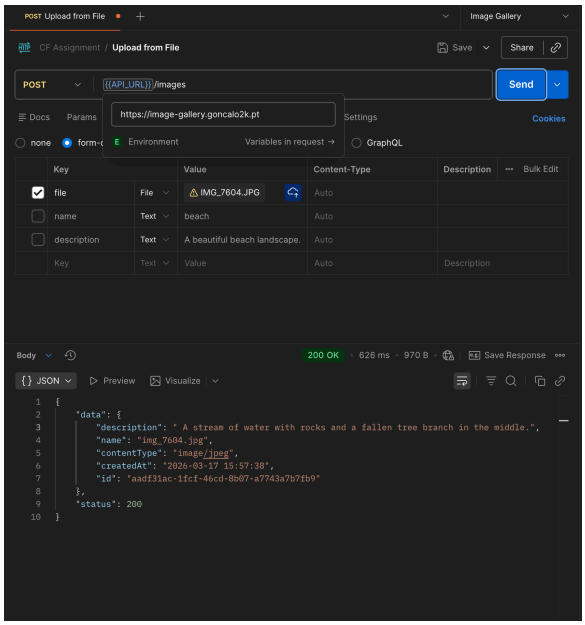


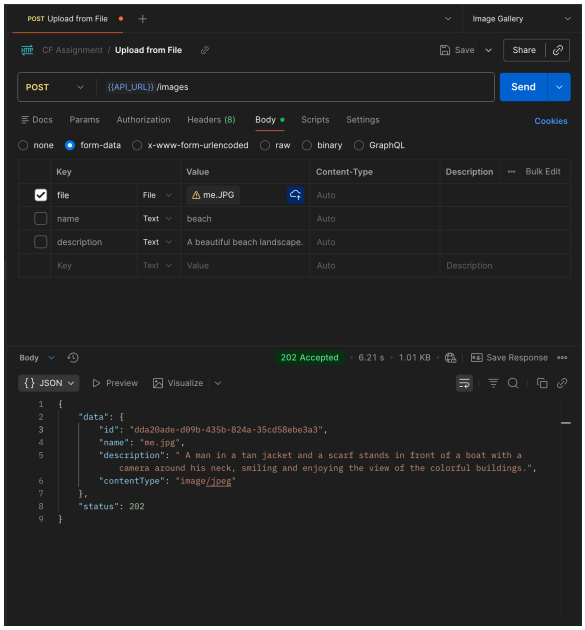


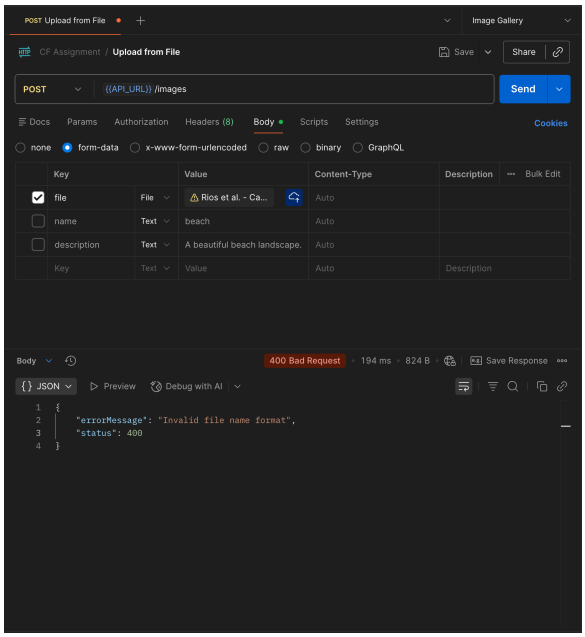
5.2 Ingesting Images

Hit `POST https://image-gallery.goncalo2k.pt/images` with fields `file`, `name?`, and `description?`. To ingest from a remote URL, POST to `/images/external` with `fileUrl`, `name?`, `description?`.

5.2.1 Testing Evidence



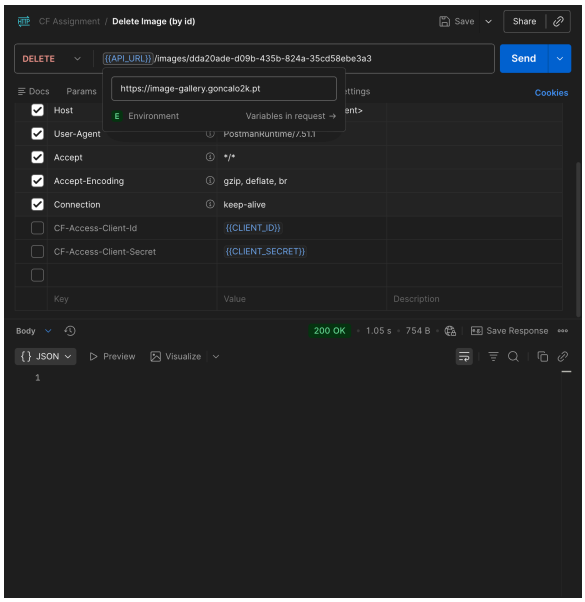


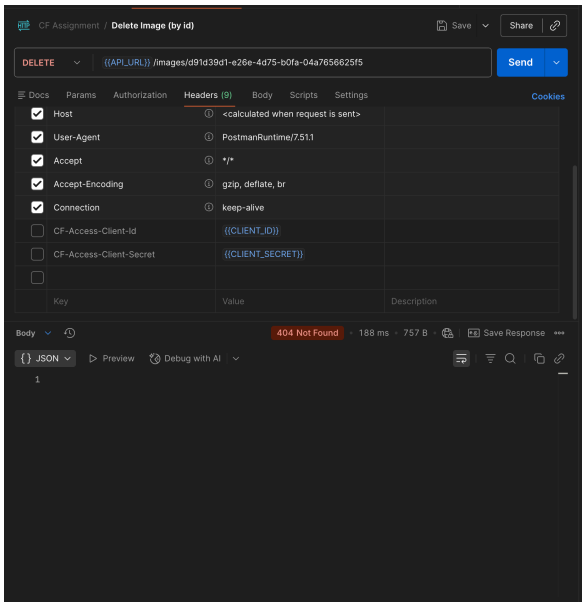


5.3 Deleting Images

Hit `DELETE https://image-gallery.goncalo2k.pt/images/:id`.

5.3.1 Testing Evidence

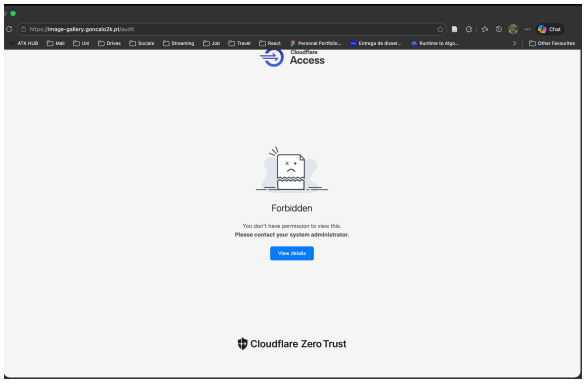


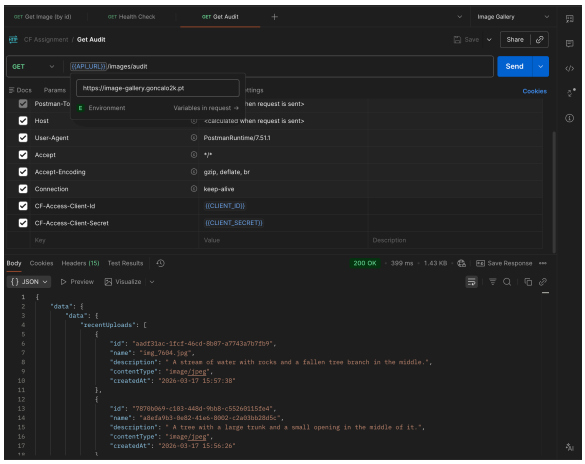


5.4 Administrative Audit Endpoint

Send `GET https://image-gallery.goncalo2k.pt/images/audit?offset=0&limit=20` with the Access headers to retrieve recent uploads plus aggregate stats (`totalImages`, `lastUpdated`). Use this endpoint to populate dashboards or monitor image insertion or deletion. Note that this endpoint is also paginated.

5.4.1 Testing evidence





6 Solution Architecture

Image gallery was built as a combination of several Cloudflare-managed services that each handle a discrete responsibility while sharing data through Worker bindings and secure

environment variables.

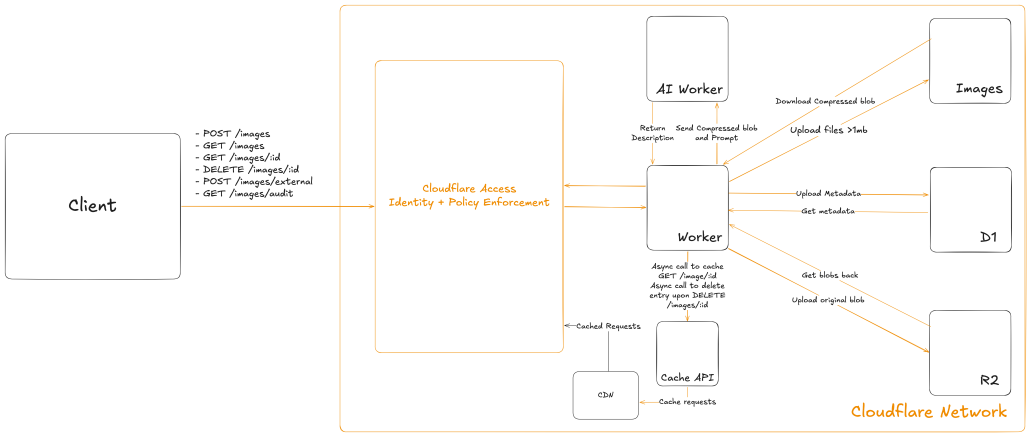
6.1 System Components and Architecture

6.1.1 System Components

- **R2 Object Storage** – Private bucket that stores the binary data for every upload. Workers never expose the bucket directly; all reads/writes go through bindings so IAM stays centralized.
- **Workers AI** – Generates descriptive alt text when the user does not provide one. Requests are throttled by resizing buffers first to stay within inference limits.
- **D1 Database** – Holds authoritative metadata (name, description, MIME type, timestamps). Lookups prevent duplicate uploads and feed `/images`, `/images/audit`, and delete checks.
- **Cache API** – Provides immutable edge caching for `GET /images/:id`. Deletes schedule cache invalidation via `executionCtx.waitUntil` to keep replicas in sync.
- **Cloudflare Access** – Protects `/images/audit` (and any future admin route) with service tokens enforced by `auth.middleware.ts`.
- **Cloudflare Images** – Acts as an on-the-fly transformer so Workers AI receives 1024×1024 JPEGs even when creators upload multi-megabyte PNGs.

6.1.2 High-level System Architecture

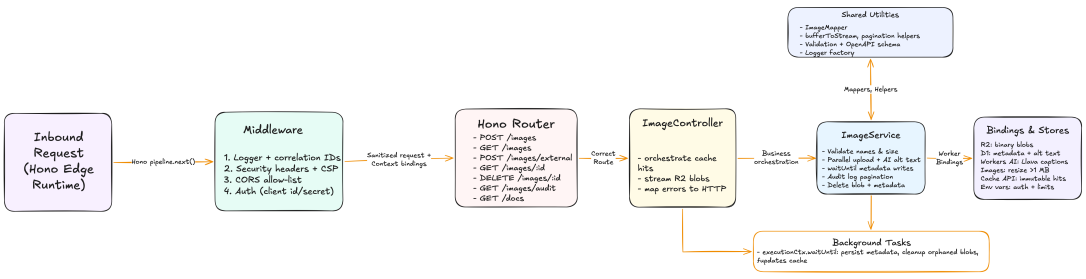
The worker sits at the center of the platform and orchestrates multiple services: R2 stores every uploaded binary, Workers AI produces fallback descriptions, D1 keeps normalized metadata, Cloudflare Images resizes large buffers before inference, and the Cache API accelerates read-heavy endpoints. Cloudflare Access service tokens wrap the privileged `/images/audit` route behind Zero-Trust authentication while environment secrets keep every credential outside the codebase. Each upload request streams the file into R2, generates or reuses alt text, and enqueues a metadata write in D1; download requests hydrate cache entries with immutable headers so subsequent requests are answered directly from Cloudflare's edge. Supporting scripts (batch upload + clear) exercise the same public endpoints which means operational automation never bypasses access controls. The diagram below captures how those building blocks interact at runtime.



6.1.3 High-level Worker Architecture

The Framework utilzed to build the worker was Hono, as it is compatible with Cloudflare out-of-the-box and very lightweight, which sped up the delivery process.

From a code perspective, `src/app.ts` wires the Hono router together with middleware for logging, CORS, authentication, and security headers so every request flows through a consistent envelope. `routes/image.routes.ts` mounts the controller methods under `/images`, while `ImageController` focuses on HTTP concerns such as parsing payloads, caching R2 responses, invalidating cache entries after deletes, and translating service results into JSON bodies. The heavy lifting lives in `ImageService`: it validates filenames and MIME types, streams uploads to R2, generates alt text with Workers AI (resizing buffers via the Images binding when necessary), inserts metadata into D1, fetches audit logs, and runs deletion fan-out across R2 and D1 in parallel. Utility modules (`image-mapper`, `utils/utils.ts`) provide reusable helpers for pagination, CSV/env parsing, and DTO transformations so controllers and services stay small. Middleware round things out: `auth.middleware.ts` protects configured routes with timing-safe header comparisons, `logger-middleware.ts` attaches correlation IDs plus structured logs, `cors.middleware.ts` enforces allow-lists, and `security-headers.middleware.ts` emits CSP/nosniff defaults. Tests under `src/**/*.spec.ts` exercise each layer, giving rapid feedback when iterating on the worker logic. This can be shown in the following diagram



7 Implementation

The creation of building blocks was developed

7.1 Prerequisites:

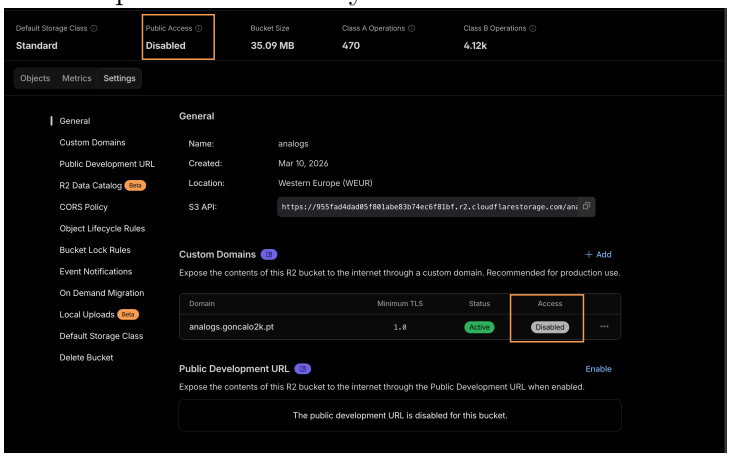
1. Sign up for a Cloudflare Account
2. Install Node.js

7.2 Worker Creation

1. Install Wrangler CLI
   `pnpm i wrangler@latest`
2. Login using Wrangler CLI
   `wrangler login`
3. Create a worker using Wrangler CLI
   `pnpm create cloudflare@latest image-gallery`
4. Select the directory where the project is going to be stored, select **Framework Starter** and then **Hono** as the framework.
5. Enable Git for version control and deploy your worker.
6. Finally, if the worker is supposed to be hosted on a custom domain, add the following lines to your `wrangler.jsonc`:
   `"routes": [{ "pattern": "image-gallery.goncalo2k.pt", "custom_domain": true } ]`

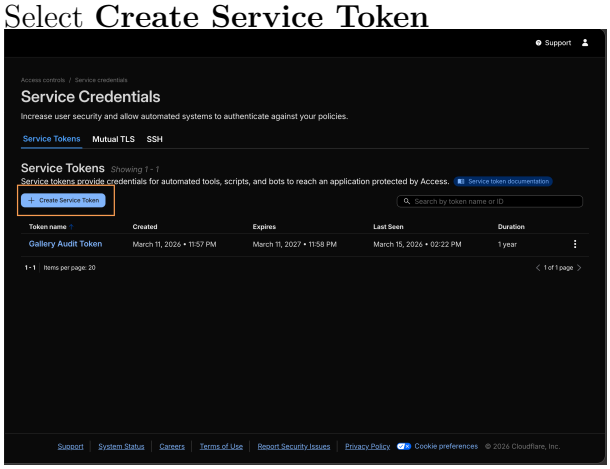
7.3 R2 and D1 Creation

1. Create D1 Database using Wrangler
   `npx wrangler@latest d1 create analogs-db`
2. Create R2 Blob Storage using Wrangler
   `npx wrangler r2 bucket create analogs`
3. Login to Cloudflare Dashboard
4. Go to the **Storage** section, look for R2 and select your bucket.
5. Disable public access for your bucket.

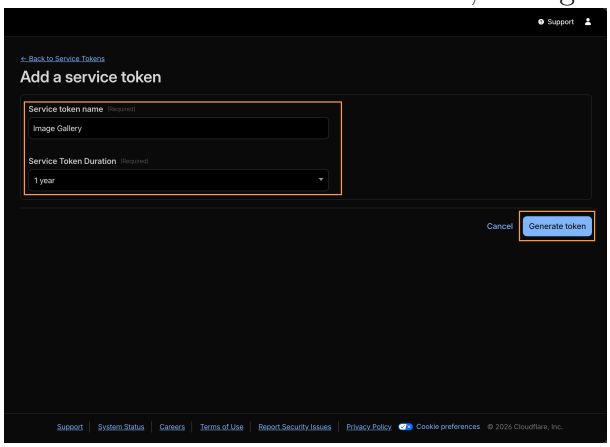


7.4 Service Token Creation

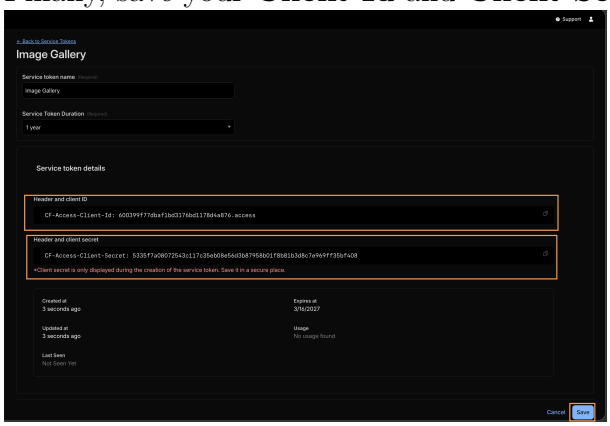
1. Login to Cloudflare Dashboard
2. Go to **Zero Trust**, **Access Controls** and then **Service Credentials**
3. Select **Create Service Token**



4. Add a name and a token duration, then generate it



5. Finally, save your **Client Id** and **Client Secret**

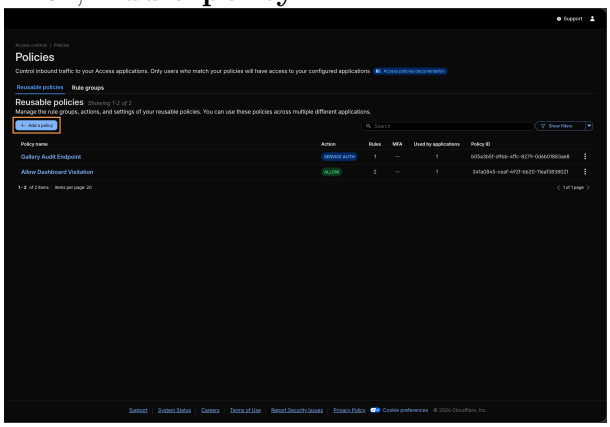


6. Then, use Wrangler to put these values in the **Secret Store**

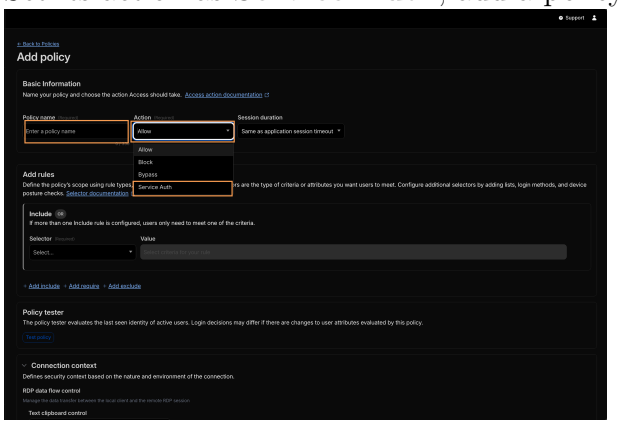
```
npx wrangler secret put CF-Access-Client-Secret
npx wrangler secret put CF-Access-Client-Id
```

7.5 Security Policy Creation

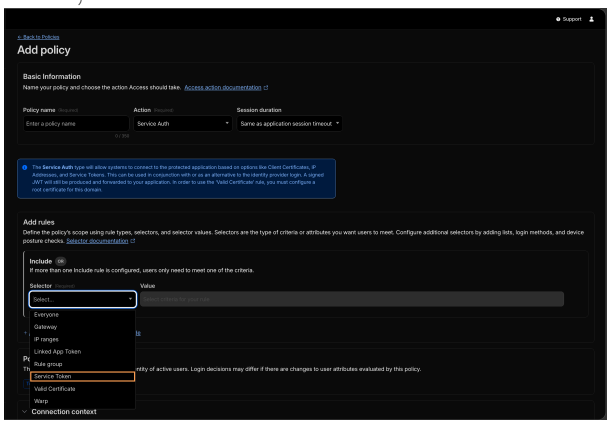
1. Login to Cloudflare Dashboard
2. Go to **Zero Trust**, **Access Controls** and then **Policies**
3. Then, **Add a policy**



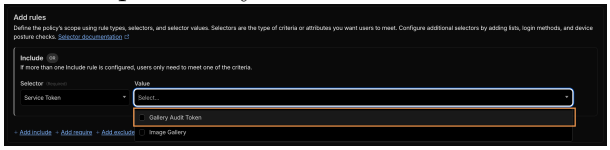
4. Set its action as **Service Auth**, add a policy name, and its session duration



5. Then, add an **include rule** and select **Service Token**

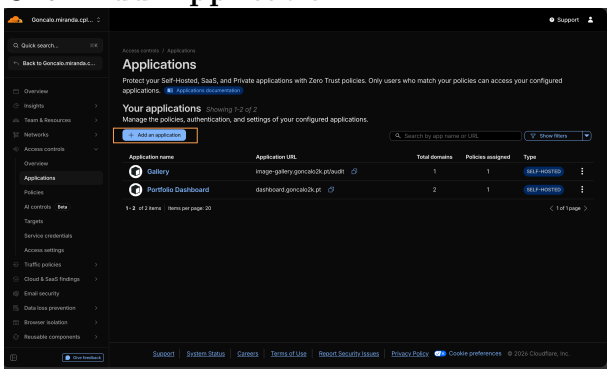


6. Use the previously created token

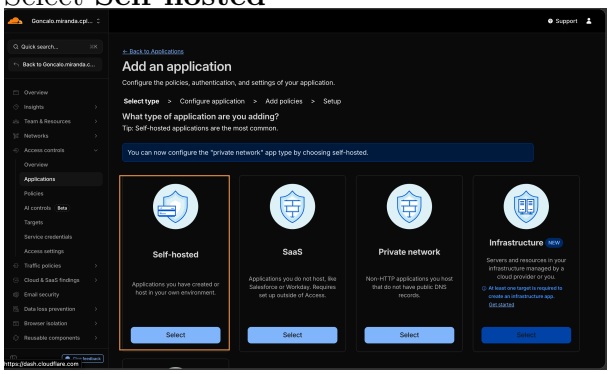


7.6 Access Policy Creation

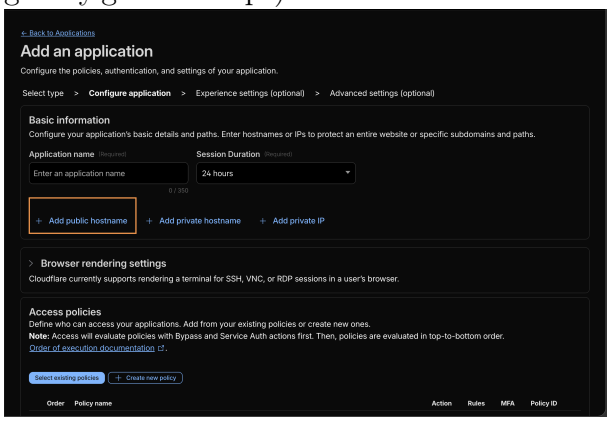
1. Login to Cloudflare Dashboard
2. Go to **Zero Trust**, **Access Controls** and then **Applications**
3. Click **Add Application**



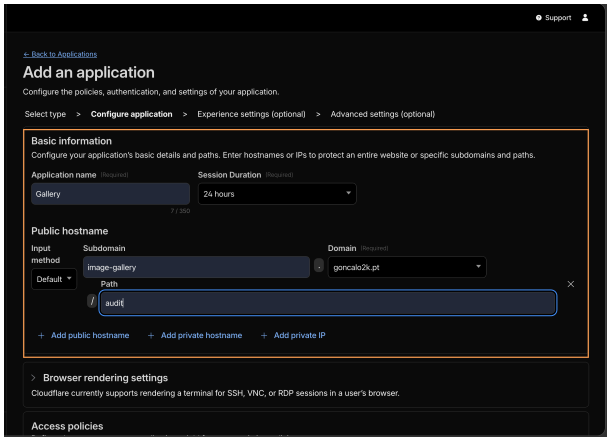
4. Select **Self-hosted**



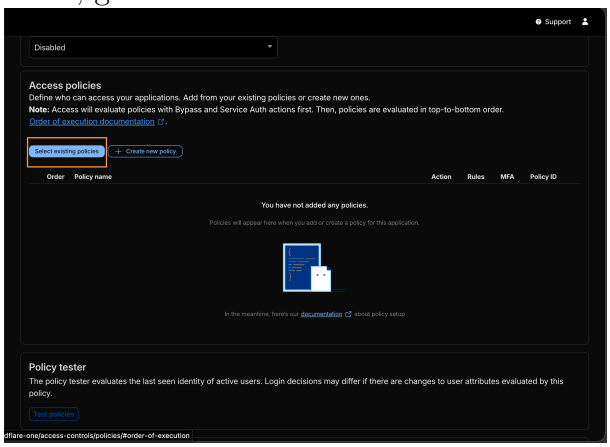
5. Add the public hostname for the app you want to protect (in this case, image-gallery.goncalo2k.pt)



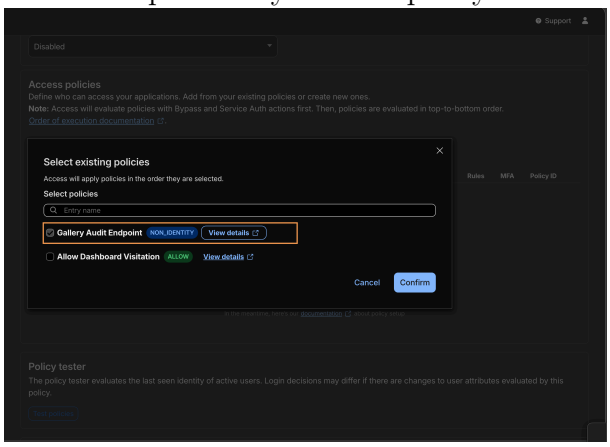
6. Add Application name, session duration, and select the intended paths and subdomains if needed.



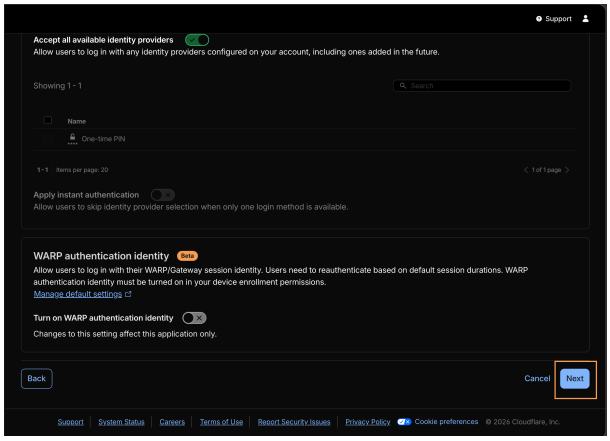
7. Then, go on to **Access Policies** section and select an existing policy



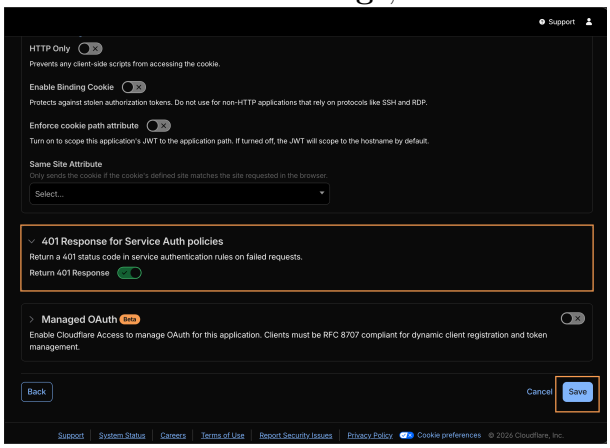
8. Select the previously created policy



9. Then hit **Next**



10. In the **Advanced Settings**, turn on **401 Response for Service Auth policies**



7.7 Worker Code Development

Each handler exists for a specific operational need:

- **`GET /health` / `/docs` / `/openapi.json`** – Remain unauthenticated to support monitoring and human documentation access. Security middleware still applies headers, but `auth.middleware.ts` exits early for these paths so dashboards and Swagger UI render without tokens.
- **`GET /images`** – Lists metadata out of D1 using `offset/limit` sanitized by `getPaginationParams`. The rationale is to keep control planes lightweight: clients can paginate without worrying about `COUNT(*)` drift, and limit clamps ($\leq$100) prevent expensive scans.
- **`POST /images`** – Accepts uploads and enforces strict invariants before spending resources. Controllers verify only one `file` part exists, `ImageService` normalizes names to lowercase (deduping by logical name instead of random IDs), and MIME validation guarantees R2 never stores unsupported formats. Returning **200** when metadata already exists keeps uploads idempotent; returning **202** when a new asset enters the system acknowledges the blob write immediately while metadata inserts run via `executionCtx.waitUntil`, ensuring the client isn't blocked by D1 latency. If that background write fails, the service deletes the blob to maintain consistency. To ensure

the image was correctly uploaded, polling this endpoint can be an option - as soon as it responds with 200, the system is sure it was successfully uploaded both to R2 and to D1. This is a relatively common pattern, seen in Github's REST API for statistics, for example.

- `POST /images/external` – Mirrors the local file upload flow but first validates the URL with `parseHttpUrl` (http/https only, host required) and checks the upstream `content-type` header after fetching with redirects disabled. This endpoint exists in order to enable dynamic ingestion of assets without uploading files directly, while still benefiting from the same dedupe and MIME guardrails.

- `GET /images/id` – Streams binaries straight from the edge cache when possible; on cache miss it loads from R2, looks up metadata in D1, and sets `ALT_HEADER_NAME`, `Content-Type`, `Cache-Control`, and an immutable `ETag`. The cache entry key is the raw request URL, so variant paths don't collide. This endpoint is the public face of the gallery, hence the extra emphasis on immutable headers and cache invalidation hooks.

- `DELETE /images/id` – Fan-outs to R2 and D1 with `Promise.allSettled`; if D1 reports `changes === 0`, the handler returns **404** and skips R2 cleanup (because nothing should exist). Successful deletions return **200** with no body, and a `waitUntil` call evicts the cached file to avoid stale responses. This keeps destructive operations fast while guaranteeing both data stores stay in sync.

- `GET /images/audit` – Protected by `auth.middleware.ts` because it exposes aggregate stats (`totalImages`, `lastUpdated`) and a paginated list of recent uploads. Same pagination helper enforces limits, and the response mirrors production monitoring needs (UI dashboards, CLI scripts). Keeping it read-only and authenticated balances observability with security.

- **Database Schema** – A single `images` table in D1 stores { `id TEXT PRIMARY KEY`, `name TEXT UNIQUE NOT NULL`, `description TEXT NOT NULL`, `content_type TEXT`, `created_at TEXT DEFAULT CURRENT_TIMESTAMP` } plus an index on `name`. That schema mirrors what the worker needs at runtime: the `name` uniqueness constraint implements deduplication for uploads, `created_at` backs audit ordering and cache `ETag`s, and `content_type` lets D1 remain the source of truth for MIME types. By keeping the table narrow this API can paginate and audit without scanning bloated rows. Besides this, the usage of an index on the `name` column enables faster queries when checking for already existing files by name (which is done on file upload).

Cross-cutting middleware reinforces these choices. `loggingMiddleware` injects/propagates `X-Correlation-Id` so every log line ties back to a single request (and downstream systems can pass their own IDs). `cors.middleware.ts` enforces allow-lists derived from `ALLOWED_ORIGINS`, returning early when the `Origin` header is disallowed. `auth.middleware.ts` consults `AUTH_ROUTES`: it skips open paths entirely, but when enabled it extracts user-provided headers (`CLIENT_ID_HEADER`, `CLIENT_SECRET_HEADER`), ensures both buffers are the same size, and then runs timing-safe equality checks so attackers can't measure response time differences. `security-headers.middleware.ts` blankets every response with CSP, `X-Content-Type-Options`, and Referrer-Policy, etc., and only relaxes CSP for `/docs` so Swagger assets can load. And even outside middleware,

ImageService includes safeguards such as rolling back R2 blobs if the metadata insert fails, which keeps storage in sync without manual cleanup. Together, these layers make the Worker predictable and safe: each endpoint is as open or locked down as intended, and any failure path still emits traceable logs.

8 Performance Considerations

The Worker architecture optimizes both latency and resource consumption:

1. **Edge caching as first hop** – ImageController.getImage hits caches.default before R2/D1 and caches successful responses with immutable TTLs and strong ETags. Deletes schedule caches.default.delete() inside executionCtx.waitUntil to prevent stale assets lingering at the edge.

2. **Parallel ingestion** – R2 uploads and Workers AI captioning run concurrently via Promise.allSettled. Deletions from R2 and D1 are also parallelized in the /DELETE /images/:id, speeding up deletions. Metadata writes upon file upload execute in executionCtx.waitUntil, so client requests only wait for blob persistence and caption generation, not D1 inserts. Finally, insertions and deletions from Cloudlfare's CDN are also done via non-blocking operations, enabling faster response times.

3. **Smart payload bounds** – MAX_UPLOAD_SIZE_MB guards uploads, while resizeImageForAI uses the Images binding to downscale >1 MB buffers to 1024×1024 JPEGs, reducing inference cost and allowing the AI models to process all images sent within the 20MB range.

4. **Efficient pagination** – getPaginationParams sanitizes offset/limit (max 100) for /images and /images/audit, avoiding large scans. Prepared statements reuse query plans, keeping D1 latency steady.

5. **Streamed delivery** – Downloaded R2 objects are wrapped in File instances and streamed to the client, so Workers can start sending bytes immediately instead of buffering entire images.

6. **Operational throttling** – Automation scripts (scripts/batch-upload.mjs, scripts/clear-all-images.mjs) expose concurrency/delay knobs, ensuring bulk jobs do not overwhelm the Worker or exceed API quotas.

9 Knowledge Gaps and Learning Process

Despite previous experience with Cloudflare Workers, several capabilities required deeper research before implementing the final solution. Each gap below lists the referenced material and the concrete takeaway that shaped the codebase.

1. **Workers AI pipeline design** – The Workers AI use-case catalog clarified how to use Workers AI and all the necessary models from Workers, what payload shapes they

expect, and how to stay within neuron budgets (capping token counts was the chosen approach). This guided the `ImageService.generateImageAltText` implementation. However, it soon became obvious that images over a few bytes would be problematic, as the AI model in use (llava-1.5-7b-hf) didn't accept such files. For this, Transform Image's worker binding was used to resize images and the smaller, resized buffers were utilized for the inference.

2. **D1 query ergonomics** – The D1 documentation explained prepared statements, result helpers (`.first()`, `.run()`), and schema deployment flows. Those patterns are used in every metadata access path. In addition, the `pnpm run setup-local-db` / `deploy-db` scripts are also based in D1 functions, by using the API's endpoints.

3. **R2 usage from Workers** – The R2 Workers API reference was key to understand how the `R2Bucket`binding works and how uploads, fetches and deletions work within a Worker's context.

4. **Typed bindings** – The Worker best-practices note on wrangler types showed how to auto-generate `worker-configuration.d.ts`, which led to the `pnpm run cf-typegen` script and the strongly typed `AppEnv` interface.

5. **Edge caching semantics** – Building `ImageController.getImage` required revisiting the Cache API docs to confirm how `caches.default.put()` behaves within `waitUntil`, what are good practices in key-creation, what cache strategies can be implemented and how cache invalidation can be done (in this case, after deletions).

6. **Execution context guarantees** – The Context API reference clarified how non-blocking operations can be leverage in Workers, via `executionCtx.waitUntil`, as this may extend a worker's lifetime for another extra 30 seconds, reinforcing the decision to offload D1 writes and cache invalidations into that lifecycle hook, enabling the API to have faster response times.

7. **Timing-safe auth** – To harden `/images/audit` and any protected routes, the Worker follows the timing-attack protection example by comparing credentials with `crypto.subtle.timingSafeEqual`, preventing side-channel leaks even when headers are incorrect.

Documenting these learnings ensures future contributors know which Cloudflare primitives influenced each architectural decision and where to dig deeper when requirements evolve.

10 Security Considerations

Security spans middleware, bindings, and operational guardrails:

1. **Authenticated admin routes** – `auth.middleware.ts` inspects `CF-Access-Client-Id` / `CF-Access-Client-Secret`, enforces timing-safe comparisons with `crypto.subtle.timingSafeEqual`, and scopes protection via `AUTH_ROUTES` so sensitive endpoints (e.g., `/images/audit`) require valid service tokens.

2. **Least-privilege bindings** – `wrangler.jsonc` binds only the required resources (R2, D1, Images, AI) plus explicit env vars. Secrets never live in code, and `.dev.vars.example` documents placeholders so local dev mirrors production without credential reuse.

3. **Request hygiene** – `cors.middleware.ts` parses `ALLOWED_ORIGINS` CSV allow-lists, rejecting disallowed origins. `security-headers.middleware.ts` adds CSP, `X-Content-Type-Options`, and `Referrer-Policy`, relaxing CSP solely for `/docs` so Swagger assets can load safely.

4. **Data consistency & cache cleanup** – Delete operations fan out to R2 + D1 via `Promise.allSettled`, and cache entries are purged asynchronously with `executionCtx.waitUntil(caches.default.delete(...))`, preventing orphaned binaries or stale public responses.

5. **Input validation** – `isValidImageName`, `MAX_UPLOAD_SIZE_MB`, and strict MIME checks on external ingestion prevent malformed uploads. FormData mappers normalize names to lowercase to avoid duplicates with different casing.

6. **Structured logging** – `loggingMiddleware` issues correlation IDs per request, and the Pino logger redacts sensitive headers, enabling auditing without exposing secrets.